

Desenvolvimento de Algoritmos para Pré-processamento de Dados

Autores: João Vitor da Conceição de Almeida
Deivid Souza dos Santos Oliveira Flávio
André Gomes Neto

Agenda

O objetivo dessa apresentação é apresentar o resultado do desenvolvimento de uma biblioteca de pré-processamento de dados, etapa realizada durante o projetos de machine learning.

1. Contextualização:

Importância do Pré-Processamento de Dados

2. Funcionalidades :

Tratamento de Dados Ausentes (MissingValueProcessor), Escalonamento de Dados Numéricos (Scaler), Codificação de Dados Categóricos (Encoder)

3. Implementação das Funcionalidades:

Exemplificação de implementações das funcionalidades a serem atendidas

4. Resultados:

Os testes fornecidos passaram e validação do métodos implementados

5. Considerações finais:

Desafios enfrentados e comparativo

Contextualização

- Etapa essencial antes da construção de modelos de aprendizado de máquina
- Transforma dados brutos em formato limpo e utilizável
- Garante precisão, consistência e relevância
- Modelos mais precisos e confiáveis



Funcionalidades



- MissingValueProcessor:

Responsável pelo tratamento de valores nulos

Métodos	Função
isna()	Identifica e retorna todas as linhas que contêm valores nulos em colunas específicas.
notna()	O inverso de isna, retorna todas as linhas que não contêm valores nulos.
fillna()	Preenche valores ausentes utilizando métodos estatísticos (mean, median, mode) ou um valor padrão.
dropna()	Remove completamente as linhas que contêm dados faltantes.

Desenvolvimento de Algoritmos para Pré-processamento de Dados

Funcionalidades



- Scaler

Responsável pelo escalonamento dos dados

Métodos	Função
<code>minMax_scaler()</code>	Normaliza os dados para um intervalo fixo (geralmente $[0, 1]$).
<code>standard_scaler()</code>	Padroniza os dados, resultando em uma distribuição com média 0 e desvio padrão 1 (Z-score).

Funcionalidades



- Encoder

Responsável por traduzir variáveis categóricas para o tipo numérico.

Métodos	Função
label_encode()	Atribui um número inteiro único para cada categoria em uma coluna.
oneHot_encode()	Cria novas colunas binárias (0 ou 1) para cada categoria, evitando a criação de uma relação de ordem artificial.

Funcionalidade vs Implementação

- **isna:**

'Entrada'

```
[  
{"cliente": "Ana", "idade": 25, "salario": 3000, "cidade":  
"São Paulo", "genero": "Masculino"},
```

```
{"cliente": "Bruno", "idade": None, "salario": 2500, "cidade":  
"Rio de Janeiro", "genero": "Feminino"},  
{"cliente": "Carlos", "idade": 40, "salario": None, "cidade":  
None, "genero": "Masculino"},  
]
```

'Saida'

```
[  
{"cliente": "Bruno", "idade": None, "salario": 2500, "cidade":  
"Rio de Janeiro", "genero": "Feminino"},  
{"cliente": "Carlos", "idade": 40, "salario": None, "cidade":  
None, "genero": "Masculino"},  
]
```

```
def isna(self, columns: Set[str] = None) -> Dict[str, List[Any]]:  
    if columns is None:  
        has_null = any(None in col_vals for col_vals in self.dataset.values())  
        return self.dataset if has_null else {}  
    else:  
        target_columns = self._get_target_columns(columns)  
        return {col: [v for v in self.dataset[col] if v is None] for col in target_columns}
```

Funcionalidade vs Implementação

- notna:

```
'Entrada'
[
  {"cliente": "Ana", "idade": 25, "salario": 3000,
  "cidade": "São Paulo", "genero": "Masculino"},
  {"cliente": "Bruno", "idade": None, "salario": 2500,
  "cidade": "Rio de Janeiro", "genero": "Feminino"},
  {"cliente": "Carlos", "idade": 40, "salario": None,
  "cidade": None, "genero": "Masculino"},
]
'Saida'
[
  {"cliente": "Ana", "idade": 25, "salario": 3000,
  "cidade": "São Paulo", "genero": "Masculino"},
]
```

```
def notna(self, columns: Set[str] = None) -> Dict[str, List[Any]]:
    return self._filter_rows(columns, lambda row: all(v is not None for v in row.values()))
```


Funcionalidade vs Implementação

- **MinMax_Scaler:**

```
'Entrada'  
{  
"idade": [22, 35, 47, 51, 62],  
"salario": [2500, 4200, 5200, 6100, 7900]  
}
```

```
'Saida'  
{  
"idade": [0.0, 0.325, 0.625, 0.725, 1.0],  
"salario": [0.0, 0.3148, 0.5, 0.6667, 1.0]  
}
```

```
def minMax_scaler(self, columns: Set[str] = None):  
    for column in self._get_target_columns(columns):  
        values = self.statistics._non_null(column)  
        self.statistics._assert_numeric(values, 'minMax_scaler')  
        if not values:  
            continue  
  
        v_min, v_max = min(values), max(values)  
        diff = v_max - v_min  
        self.dataset[column] = [  
            0.0 if diff == 0 and v is not None else ((v - v_min) / diff if v is not None else None)  
            for v in self.dataset[column]  
        ]
```

Funcionalidade vs Implementação

- **Standard_Scaler:**

```
'Entrada'
{
"idade": [22, 35, 47, 51, 62],
"salario": [2500, 4200, 5200, 6100, 7900]
}

'Saida'
{
"idade": [-1.556, -0.611, 0.262, 0.553, 1.353],
"salario": [-1.480, -0.541, 0.011, 0.508, 1.502]
}
```

```
def standard_scaler(self, columns: Set[str] = None):
    for column in self._get_target_columns(columns):
        values = self.statistics._non_null(column)
        self.statistics._assert_numeric(values, 'standard_scaler')
        if not values:
            continue

        mu, sigma = self.statistics.mean(column), self.statistics.stdev(column)
        self.dataset[column] = [
            None if v is None else (0.0 if sigma == 0 else (v - mu) / sigma)
            for v in self.dataset[column]
        ]
```

Funcionalidade vs Implementação

- **Label_encode:**

```
'Entrada'
{
"cidade": ["Salvador", "Feira", "Salvador", "Ilhéus", "Feira"],
"genero": ["Feminino", "Masculino", "Feminino", "Feminino",
"Masculino"]
}
'Saida'
{
"cidade": ["Salvador", "Feira", "Salvador", "Ilhéus", "Feira"],
"genero": ["Feminino", "Masculino", "Feminino", "Feminino",
"Masculino"],
"cidade_label": [0, 1, 0, 2, 1],
"genero_label": [0, 1, 0, 0, 1]
}
```

```
def label_encode(self, columns: Set[str]):
    self._validate_columns(columns)
    for column in columns:
        values = self.statistics._non_null(column)
        mapping = {cat: idx for idx, cat in enumerate(sorted(set(values)))}
        self.dataset[column] = [mapping[v] if v is not None else None for v in self.dataset[column]]
```

Funcionalidade vs Implementação

- **oneHot_encode:**

```
'Entrada'
{
"cidade": ["Salvador", "Feira", "Salvador",
"Ilhéus", "Feira"],
"genero": ["Feminino", "Masculino", "Feminino",
"Feminino", "Masculino"]
}
```

```
'Saida'
{
"cidade": ["Salvador", "Feira", "Salvador",
"Ilhéus", "Feira"],
"genero": ["Feminino", "Masculino", "Feminino",
"Feminino", "Masculino"],
"cidade_Salvador": [1, 0, 1, 0, 0],
"cidade_Feira": [0, 1, 0, 0, 1],
"cidade_Ilhéus": [0, 0, 0, 1, 0],
"genero_Feminino": [1, 0, 1, 1, 0],
"genero_Masculino": [0, 1, 0, 0, 1]
}
```

```
def oneHot_encode(self, columns: Set[str]):
    self._validate_columns(columns)
    for column in columns:
        values = self.statistics._non_null(column)
        for cat in sorted(set(values)):
            self.dataset[f"{column}_{cat}"] = [1 if v == cat else 0 for v in self.dataset[column]]
        del self.dataset[column]
```

Resultados

- Todos os Testes passaram

```
• PS C:\Users\jv425\OneDrive\Documentos\GitHub\tia-lu-preprocessing-salvador> python -m unittest test_preprocessing.py
.....
-----
Ran 12 tests in 0.009s

OK
○ PS C:\Users\jv425\OneDrive\Documentos\GitHub\tia-lu-preprocessing-salvador> █
```

Considerações finais



- O sucesso do modelo depende da qualidade dos dados
- O pré-processamento é **indispensável** para aprendizado confiável
- Implementações em **Python puro**, sem bibliotecas externas
- Base para construção de sistemas de recomendação de alta performance